ECEN5743 Final Project

Yash Trivedi

1. Introduction

Heat transfer is an important area of research as the management of energy is often a limiting design factor in many engineering systems. Typically, engineering systems rely on single phase forced air or liquid thermal management solutions. However, required heat loads and heat fluxes continue to grow making single phase cooling ineffective and infeasible. This has led researchers to explore two-phase cooling situations which can make use of available latent heat of vaporization. Tapping into the additional energy potential leads to heat transfer capabilities improving by orders of magnitude, but multiphase heat transfer has been very difficult to model.

Two-phase heat transfer is strongly linked with single phase conductive and convective energy mechanisms, but also with interfacial liquid-vapor phenomena which introduce complicated multiscale physics. In addition, bubble nucleation is highly stochastic and non-linear as formation depends on surface properties and flow conditions. All the competing factors make the process of boiling incredibly challenging to model, and that has led to researchers turning to data-driven methods.

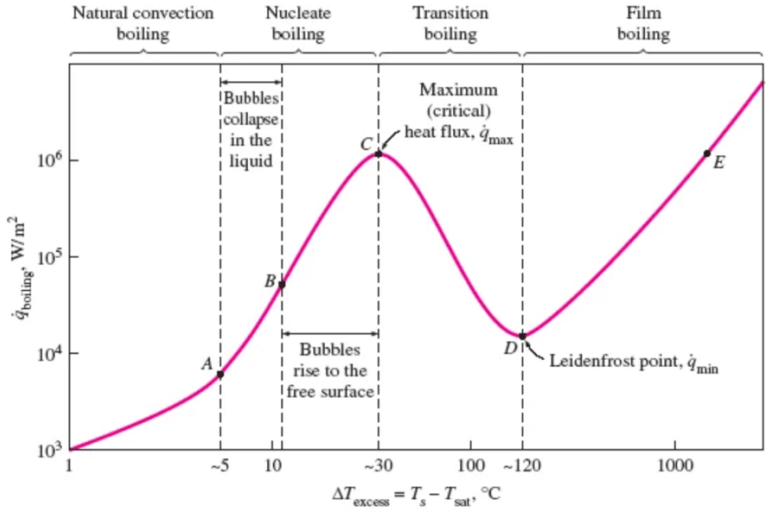

Figure 1 - Boiling curve for water at atmospheric pressure (Edge, 2026)

Furthermore, boiling is split into multiple regions depending on the heat input and surface superheat, as shown in Figure 1. Boiling starts with natural convection boiling before any bubbles begin to nucleate, and natural convection currents dominate heat transfer. Next, nucleate boiling begins where bubbles begin to form, grow, and detach from the surface. Moving through nucleate boiling, bubble formation rates and sizes continue to increase with heat input resulting in very unstable boiling. The top of the curve is signified with critical heat flux (CHF), at which point heat transfer diminishes quickly. CHF is the point where air pockets begin to form at the boiling surface, without being rewetted with liquid, causing an insulating vapor layer forming. The vapor layer destroys heat transfer ability resulting in sudden surges in temperature, from point C to E, which lead to system failure. Therefore, it is important to avoid reaching CHF and transitioning into transition or

film boiling, but heat transfer systems would benefit from operating reliably in the latter nucleate boiling regimes.

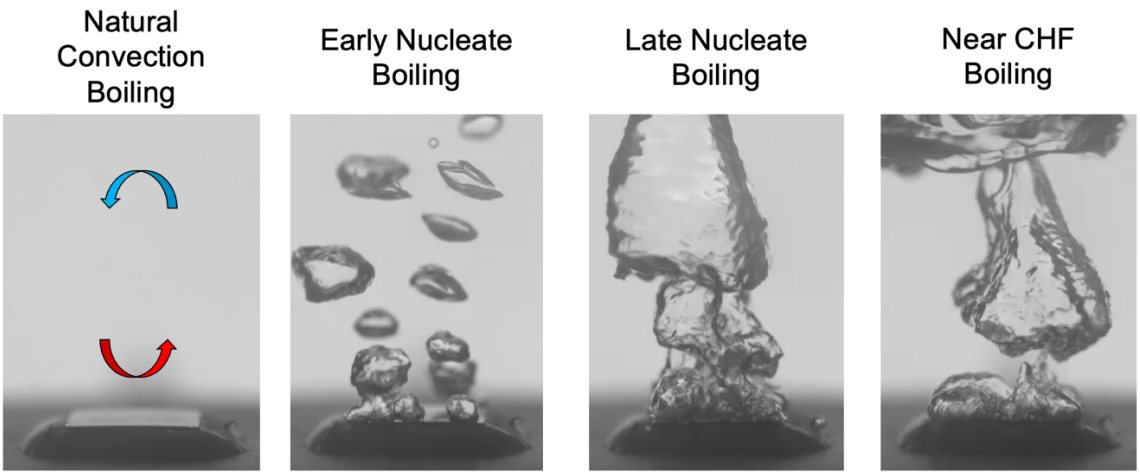

Figure 2 - Visual of different boiling regimes (Pandey et al., 2025a)

In addition, boiling is a visual process. Shown in Figure 2, the different boiling regimes have clear visual differences which has led to the research interest of applying convolution neural networks (CNNs) to heat transfer modelling. Therefore, this project will aim to use a thermal and visual dataset of pool boiling combined with a CNN architecture to make heat transfer predictions.

$$\Pi_{CHF} = \frac{T_{wall} - T_{sat}}{(T_{wall} - T_{CHF})_{CHF}}$$

As a prediction target, we define a new dimensionless variable called “CHF Proximity”, outlined in the equation above. The variable was created to give an approximation of proximity to a failure condition in a boiling system, which can be used as a design tool. The variable simply acts as a ratio between surface temperature superheat from the fluid’s saturation temperature at a given point compared to at CHF.

2. Project Schedule

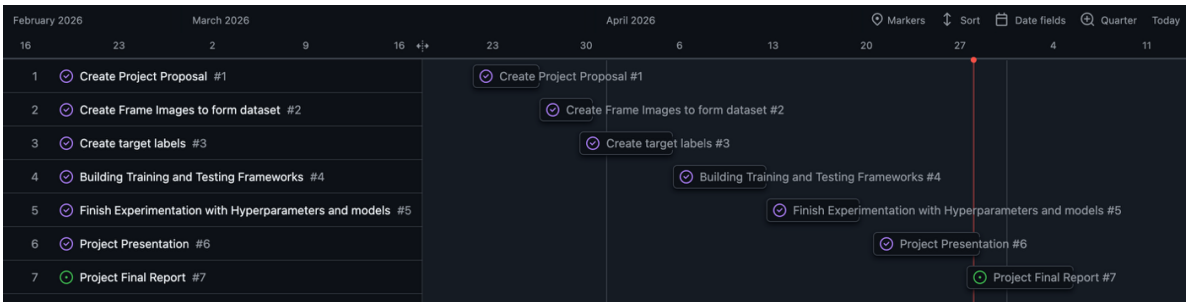

Figure 3 - Project Schedule

Figure 3 shows a screenshot from the GitHub project management tool outlining the different tasks and timelines and assigned for them. Three main deadlines were provided:

initial project proposal on 3/26, project presentation on 4/28, and final project report on 5/5. Between the initial proposal and presentation, several tasks were made to span the month, which involved preparing the dataset with images and labels, then building the test and train frameworks, and then experimenting with different hyperparameters and pretrained models. Following the completion of the code and presentation, the creation of this final report will start and finish by the final deadline of 5/5.

3. Description of Dataset

In this final project, the dataset used was provided from a published study of boiling hysteresis (Pandey et al., 2025a). The study aimed to investigate the hysteresis effect in boiling as power was ramped up to CHF, and then ramped back down (Pandey et al., 2025b). The study relied on visual and audio-based data, combined with raw temperature data, to attempt to make findings. Following the study, the group published the datasets which was very useful as boiling imaging datasets are quite rare to find. As a note, the full videos are not in the GitHub site due to size restrictions.

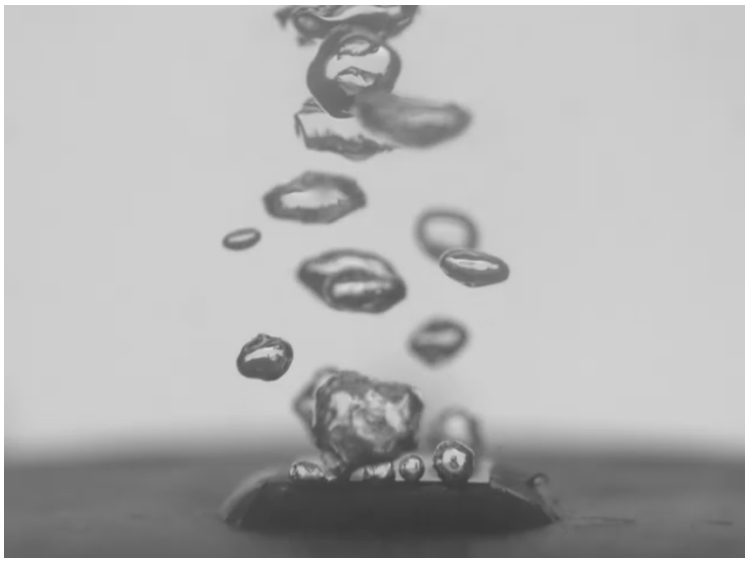

Figure 4 - Example frame from video dataset

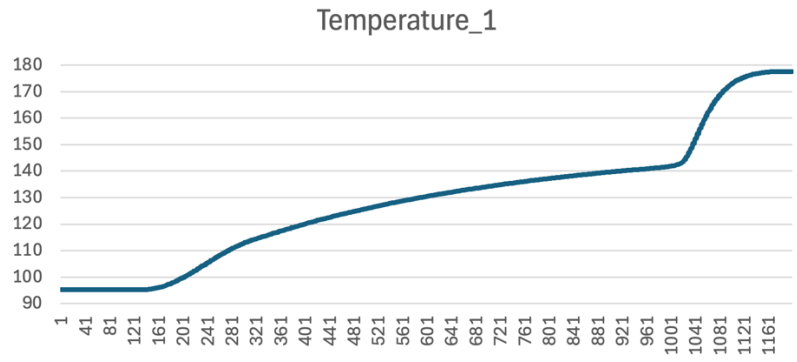

Figure 5 – Down-sampled Temperature Data from Boiling-84

An example frame that was extracted from the dataset is shown in Figure 4. The original data contained two full videos from high speed imaging, as well as full temperature data files, shown in Figure 5. The datasets were named “Boiling-84” and “Boiling-91” and contain similar boiling behavior and data. The first set was collected from boiling on a copper foam surface, and the second was from boiling off a smooth copper surface. The data was collected at the data acquisition sampling rate, which was far too high, so it was downsampled to a reasonable level. As a result, the final dataset which was used in this project contained two sets of 1200 data points spanning from beginning till just past CHF.

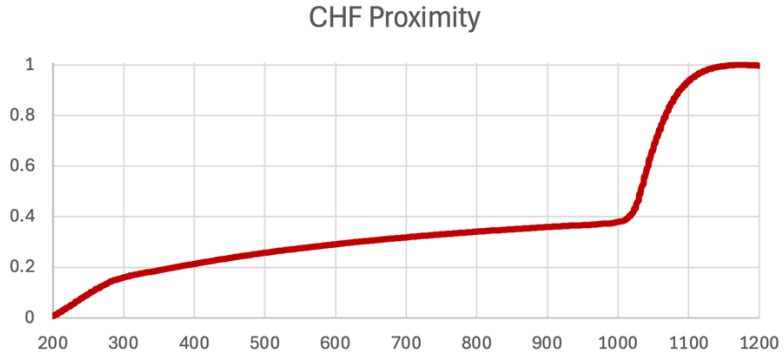

Figure 6 - Plot of CHF Proximity

The CHF proximity was then calculated for each point using the prior equation, which was plotted as shown in Figure 6. In addition, the videos were split into frames which were synchronized with the temperature data points. This created a full 2 datasets of 1200 data points with images and labels. The shape of the plot very closely follows the temperature plot as the variables are closely related.

4. Description of Network and Training Algorithm

For this project, the decision to use a pretrained CNN model was made due to the lack of available training data. It was unlikely that only 1200 datapoints would be sufficient to train a strong network. The search for a suitable pretrained model began through a literature search to check what models have worked for boiling imaging. Paz et al., conducted a study investigating how different pretrained CNN architectures would do with a boiling classification task (Paz et al., 2025). As mentioned earlier, boiling takes place in multiple regimes, and it is very useful to analyze a system and identify what regime boiling in taking place in.

CNN Architecture	Model F1 Score
Basic CNN	91.11%
AlexNet	**98.87%**

ResNet	**96.50%**
InceptionNet	94.23%

Table 1 – Summarized boiling classification F1-score (Paz et al., 2025)

As shown in Table 1, the investigation revealed that AlexNet and ResNet were the most successful in classifying the images. Therefore, both of those networks were shortlisted and tested on the dataset from this project.

CNN Architecture	MAE	RMSE	R^2	Correlation
AlexNet	0.135	0.236	0.429	0.789
ResNet-18	**0.113**	0.219	0.505	0.792
ResNet-34	0.139	0.244	0.388	0.737
EfficientNet	0.126	**0.217**	**0.514**	**0.796**

Table 2 - Testing different networks on project dataset

Table 2 shows the results of experimenting with 4 different networks. Contrary to the expectations, AlexNet was one of the worst performers. Two different ResNet models were tested, but the results showed that the lighter 18-layer model performed better. This is likely because the larger model overfit the small dataset in training. Finally, EfficientNet fell short on MAE, but the RMSE and correlation numbers beat ResNet-18. However, ResNet-18 was chosen as the project network as MAE was the performance metric.

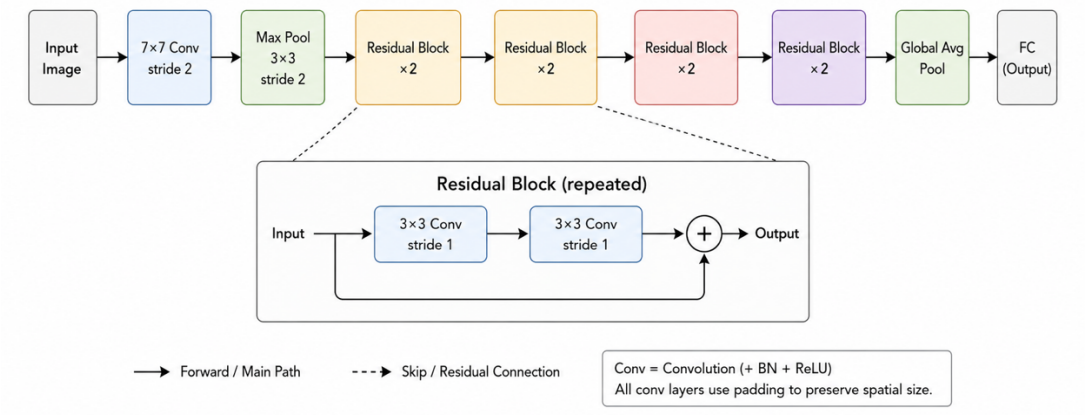

Figure 7 - ResNet-18 Architecture Diagram

ResNet-18, shown in Figure 7, was created by He et al as a solution to tackle the vanishing gradient problem with deep CNNs (He et al., 2016). It was observed that there

came a point where simply adding more convolution layers ended up hurting a model's prediction accuracy as the gradient would become increasingly small at early layers and weights would never be significantly updated. Additionally, the paper states that, besides vanishing gradient, training error can simply increase as accuracy gets saturated. They proposed a solution using "skip" or "residual" connections to jump convolution layers.

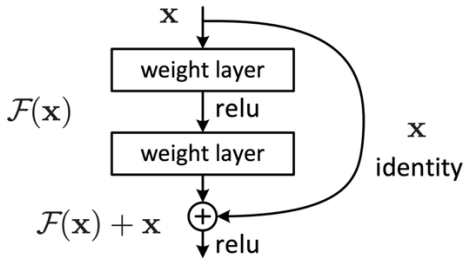

Figure 8 - Residual Learning Building Block (He et al., 2016)

The hypothesis proposed was that it would be easier for a model to train and predict the error between input and output as the residual mapping is easier to optimize than the unreferenced original mapping in typical connections, shown in Figure 8. This hypothesis resulted in a very successful model which does very well when predicting on the ImageNet dataset that it was trained on.

The original ResNet-18 architecture is trained to classify into 1000 different groups, so the output layer had to be changed to fit the regression task in this project. The original output layer was replaced with a fully connected layer with one output neuron acting as the regression head. A sigmoid activation function was applied to bound values in range.

5. Experimental Setup

Once the ResNet-18 architecture was chosen, a framework for testing and training was created to experiment and tune the model. The training and testing functions share a lot of similarities, but the training file will be used for deeper description. The code was implemented using the PyTorch modules due to personal familiarity. The performance of the network was judged using mean absolute error (MAE).

Beginning with the data loading, the dataset was split into a 50/50 test and train split. As mentioned, there were two similar datasets available. To ensure that all regimes of boiling were exposed in training and testing, it was not possible to take a larger split for training. A class was created to handle data loading and handling. The class receives and stores the address of the annotation file and image folder. Then the "CHF Proximity" column from the annotation file is copied and values are capped between 0 and 1. An index is assigned which then links the label and image together. When the image is loaded in, it is converted to RGB, matching ResNet requirements, and assigned a target label. Then, the image has any transforms applied which are outlined individually for training or test and validation. For full details, see Appendix A for the full code snippet of the Dataset class.

```python
# Set transformations for training data: resize, affine, colorJitter, normalize for ResNet18
train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomAffine(
        degrees=0,
        translate=(0.02, 0.02),
        scale=(0.98, 1.02),
    ),
    transforms.ColorJitter(brightness=0.1, contrast=0.1),
    transforms.ToTensor(),
    # ImageNet normalization values
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])
```

Figure 9 - Image augmentation applied to training set

For testing and validation, only the required transformations were applied, which include image resizing and RGB normalization for the ResNet-18 weights. In addition to those, different types of augmentation were tested to improve generalization, shown in Figure 9. The first method was color jitter to add brightness variation to the images. This resulted in a minor improvement in MAE. The next addition was random translation and scaling, implemented with the "RandomAffine" method. This would simulate bubbles of slightly different scale and position, and it improved generalization to the test set notably as MAE improved from 0.13 to 0.11. Other methods of augmentation were deemed unnatural for the dataset.

```python
class ResNet18Regressor(nn.Module):  1 usage  ytrivedi315
    # Load pretrained ResNet-18 backbone
    def __init__(self, pretrained: bool = True):  ytrivedi315
        super().__init__()
        weights = ResNet18_Weights.DEFAULT if pretrained else None
        self.backbone = models.resnet18(weights=weights)
        in_features = self.backbone.fc.in_features
        # Replace final classification layer with regression head
        self.backbone.fc = nn.Sequential(
            nn.Dropout(p=0.2),
            nn.Linear(in_features,  out_features: 1),
            nn.Sigmoid(),
        )

    def forward(self, x):  ytrivedi315
        return self.backbone(x)
```

Figure 10 - ResNet-18 Model Class

As shown in Figure 10, the ResNet-18 training architecture was implemented by loading in the backbone and pretrained weights. The original output classifier layers were replaced with a fully connected layer with one output neuron to perform regression, with a sigmoid activation function. Dropout of 20% was also applied to improve generalization, with the setting also experimented with.

```
def main():  1 usage   ytrivedi315
    # Set up argparse to control training from program call in terminal
    parser = argparse.ArgumentParser(description="Train ResNet-18 on case 84 for CHF proximity regression.")
    parser.add_argument( *name_or_flags: "--annotations", type=str, required=True, help="Case 84 aligned CSV/XLSX path")
    parser.add_argument( *name_or_flags: "--image_dir", type=str, required=True, help="Directory of extracted case 84 frames")
    parser.add_argument( *name_or_flags: "--output_dir", type=str, default="outputs_case84_resnet18")
    parser.add_argument( *name_or_flags: "--epochs", type=int, default=10)
    parser.add_argument( *name_or_flags: "--batch_size", type=int, default=32)
    parser.add_argument( *name_or_flags: "--lr", type=float, default=1e-4)
    parser.add_argument( *name_or_flags: "--weight_decay", type=float, default=1e-4)
    parser.add_argument( *name_or_flags: "--val_split", type=float, default=0.2)
    parser.add_argument( *name_or_flags: "--num_workers", type=int, default=4)
    parser.add_argument( *name_or_flags: "--seed", type=int, default=42)
    parser.add_argument( *name_or_flags: "--freeze_backbone_epochs", type=int, default=0)
    args = parser.parse_args()
```

Figure 11- Implementation of ArgParse structure

To allow for easy experimentation with the network and its hyperparameters, argparse was implemented to allow a user to customize various settings, such as batch size or learning rate, from the terminal when calling the training or testing programs, shown in Figure 11. The sweet spot for batch size was found to be 32. Learning rate was optimal at 1e-4 as larger and smaller settings were less successful at reaching a minima in training and validation loss. The weight decay parameter was added, paired with the Adam optimizer, to implement regularization and penalize large weights, and its optimal setting was 1e-4. Finally, the last key implementation was the "freeze_backbone_epochs" parameter which allows the user to only train the output layer for several epochs to reduce overfitting. However, that setting was found optimal at zero as the full backbone needed the initial epochs to adjust weights from identifying the ImageNet dataset to the boiling dataset shown.

In testing, the only differences in code were that pretrained weights were not loaded and no weight updates were performed. The checkpoint with the lowest validation MAE was saved in training and loaded in for testing. As mentioned previously, no data augmentation beyond resizing and RGB normalization is applied to the testing set.

6. Results

Performance Metric	
MAE	0.113
RMSE	0.219
R^2	0.505
Correlation	0.792

Table 3 - Final Performance Metrics

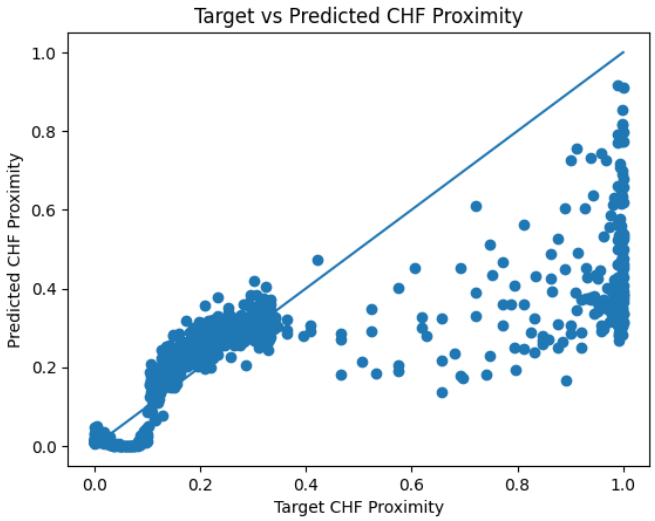

Figure 12 - Plot of Target vs Predicted CHF Proximity from best ResNet-18 run

Table 3 presents the final performance metrics that were achieved after experimentation in this project. As shown, the network achieved a minimum mean absolute error of 0.113 which was promising. However, the poor correlation numbers raised concerns, and they can be visualized in Figure 12. The plot of target against predicted CHF proximity very clearly shows that the model was performing well between values of 0.1 to 0.4 but significantly underpredicting for higher values above 0.5. This is likely due to the imbalance of training data as the majority of the dataset lies in the 0.1 to 0.4 range, as shown in Figure 6. Similar plots were created for competing networks that were tested in this project, and they exhibited similar behavior where the underprediction was sometimes more significant. This is an important shortcoming to highlight as the region with high CHF proximity is exactly where strong prediction ability is required.

In addition, there appears to be poor accuracy at low CHF proximity values below 0.1 where the model consistently predicts near zero values. However, this result is due to the temperature difference being insufficient to cause bubble formation, and boiling existing in the natural convection regime. Therefore, the frames are visually indistinguishable from no boiling taking place until bubbles begin to nucleate.

7. Summary and Conclusions

To conclude this report, this project introduced a methodology using pretrained CNN networks to identify proximity to critical heat flux in a visual boiling dataset. A dataset of boiling high speed visualization and temperature data was provided from a publication. The data was processed to create two training and testing sets of equal length and similar behavior. Through literature review and local experimentation, the ResNet-18 backbone architecture was selected for its superior performance. The final output layer was replaced with a single neuron fully connected layer which included dropout and sigmoid activation. The main hyperparameters of the network were tuned, and weight decay regularization was included to penalize large weights. Beyond standard resizing and normalization, color jitter, translation, and scaling augmentation were applied to training images to improve generalization.

As a result, the final model was able to achieve a MAE of 0.113 and RMSE of 0.219. The R^2 value was 0.505 and correlation was 0.792. Simply in terms of mean error, the results looked promising, especially with the limited dataset. However, the correlation numbers reveal the true shortcomings. The network showed strong prediction ability between a CHF proximity of 0.1 and 0.4, but very poor performance outside of that range. Above 0.5, the network significantly underpredicted CHF proximity due to the lack of data in that target range. On the other end, under 0.1, the network consistently predicted near zero values due to indistinguishable visual features in the natural convection regime where no bubbles are present yet.

To address the prediction shortcomings, it is hypothesized that the implementation of an input weighting protocol based on CHF proximity regimes would be beneficial. This would hopefully allow the training phase to update weights with more strength when handling rare, but key, data points at high CHF proximity. Furthermore, this project and network would have benefitted from a larger dataset as most neural networks would struggle to generalize well when only exposed to one test case. Finally, during the presentation a good suggestion was made to try a linear or ReLU activation function for the fully connected output layer. Due to the characteristics of the sigmoid function, it is likely that high and low CHF proximity targets, near 0 and 1, were saturating the activation function which could have contributed to the poor performance. These would be important features to investigate in future work.

8. References

Edge, E. (2026). *Water Boiling Graph Curve at 1 Atmosphere*. Engineers Edge. https://www.engineersedge.com/heat_transfer/water_boiling_graph_curve_13825.htm

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778. https://doi.org/10.1109/CVPR.2016.90

Pandey, H., Li, C., Dunlap, C., & Hu, H. (2025a). *Data from: Unveiling hysteresis of transient boiling: A multimodal perspective* [Dataset]. Dryad. https://doi.org/10.5061/dryad.ksn02v7h2

Pandey, H., Li, C., Dunlap, C., & Hu, H. (2025b). Unveiling hysteresis of transient boiling: A multimodal perspective. *Applied Thermal Engineering*, *262*, 125259. https://doi.org/10.1016/j.applthermaleng.2024.125259

Paz, C., Cabarcos, A., Concheiro, M., & Conde-Fontenla, M. (2025). Classification of boiling regimes, fluids, and heating surfaces through deep learning algorithms and image analysis. *International Journal of Heat and Mass Transfer*, *242*, 126829. https://doi.org/10.1016/j.ijheatmasstransfer.2025.126829

9. Appendices

Appendix A: Dataset Loading and Handling Class

```python
class CHFFrameDataset(Dataset):  3 usages  ytrivedi315 *
    """
    Dataset for single-frame CHF proximity regression.

    """

    def __init__(self, annotations_path: str, image_dir: str, transform=None):  ytrivedi315
        self.annotations_path = Path(annotations_path)
        self.image_dir = Path(image_dir)
        self.transform = transform

        if not self.annotations_path.exists():
            raise FileNotFoundError(f"Annotations file not found: {self.annotations_path}")
        if not self.image_dir.exists():
            raise FileNotFoundError(f"Image directory not found: {self.image_dir}")

        if self.annotations_path.suffix.lower() == ".csv":
            self.df = pd.read_csv(self.annotations_path)
        else:
            self.df = pd.read_excel(self.annotations_path)

        required_target_col = "CHF Proximity"
        if required_target_col not in self.df.columns:
            raise ValueError(
                f"Expected target column '{required_target_col}' not found. "
                f"Available columns: {list(self.df.columns)}"
            )
```

```
# Create internal frame filenames
self.df["frame_filename"] = [f"frame_{i:04d}.png" for i in range(len(self.df))]
self.mode = "filename"

self.df = self.df.copy()
# Clip CHF Proximity values between 0 and 1
self.df[required_target_col] = self.df[required_target_col].clip( lower: 0.0,  upper: 1.0)

def build_image_name(row):   ytrivedi315
    if self.mode == "filename":
        return str(row["frame_filename"])
    elif self.mode == "frame_index":
        return f"frame_{int(row['frame_index'])}.png"
    elif self.mode == "frame_index_round":
        return f"frame_{int(row['frame_index_round'])}.png"
    else:
        raise RuntimeError("Unknown dataset mode.")

# Check for missing images
exists_mask = [
    (self.image_dir / f"frame_{i:04d}.png").exists()
    for i in range(len(self.df))
]
exists_mask = np.array(exists_mask)
```

```
        missing_count = int((~exists_mask).sum())
        if missing_count > 0:
            print(f"Warning: dropping {missing_count} rows with missing image files.")
        self.df = self.df[exists_mask].reset_index(drop=True)

        if len(self.df) == 0:
            raise RuntimeError("No valid samples remain after checking image files.")

    def __len__(self) -> int:  ytrivedi315
        return len(self.df)

    def __getitem__(self, idx: int):  ytrivedi315
        # Loads an image-label pair
        row = self.df.iloc[idx]
        image_name = str(row["frame_filename"])

        # Open image and convert to RGB, clamp between 0 and 1
        img_path = self.image_dir / image_name
        image = Image.open(img_path).convert("RGB")
        target = float(row["CHF Proximity"])
        target = max(0.0, min(1.0, target))

        if self.transform is not None:
            image = self.transform(image)

        return image, torch.tensor( data: [target], dtype=torch.float32)
```